

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Knowledge Representation Design
Assessment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO3: *Implement propositional and first-order logic using resolution, unification, and inference techniques for knowledge representation and reasoning. (BTL 3)*

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 60 Mins

No. of Questions: 5

Annexure A
Knowledge Representation Design Assessment

Code of Conduct:

I G. Poojitha (192425337) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G. Poojitha

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Engineering and Knowledge Base Design	15%	
3. Propositional Logic Representation	10%	
4. First-Order Logic Representation	15%	
5. Unification and Resolution	15%	
6. Forward and Backward Chaining	15%	
7. Inference and Reasoning Accuracy	10%	
8. System Architecture / Representation Diagram	5%	
9. Prototype Implementation and Results	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE QUESTION

1. Smart Healthcare Diagnosis System

A hospital wants to develop an AI-based system that can identify possible illnesses from patient symptoms. The system contains facts such as:

- Ravi has fever.
- Ravi has cough.
- Ravi has body pain.
- Patients with fever and cough may have flu.
- Patients with flu and body pain may have viral infection.
- Patients with viral infection should be advised to rest.

Task:

1. Represent the given knowledge using **Propositional Logic**.
2. Convert the knowledge into **First-Order Logic (FOL)**.
3. Construct a knowledge base containing facts and rules.
4. Demonstrate **unification** for at least two predicates.
5. Use **forward chaining** to determine Ravi's condition.
6. Use **backward chaining** to prove whether Ravi needs rest.
7. Demonstrate **resolution** for one selected query.
8. Explain the final inference produced by the system.

2. Intelligent Loan Approval System

A bank wants to develop an AI system to determine whether a customer is eligible for a loan. The knowledge base contains information about income, credit score, employment, existing loans, and repayment history.

For example:

- Arun has a stable job.
- Arun has a high credit score.
- Arun has sufficient income.
- Customers with stable employment and sufficient income are eligible for preliminary approval.
- Customers with high credit scores and good repayment history are considered low-risk.
- Low-risk customers with preliminary approval can be recommended for loan approval.

Task:

1. Identify the **entities, facts, predicates, and rules** involved.
2. Represent the knowledge using **First-Order Logic**.
3. Build a knowledge base with at least **10 facts and 8 rules**.
4. Demonstrate **unification and substitution**.

5. Apply **forward chaining** to determine whether Arun can be recommended for approval.
6. Apply **backward chaining** for the query `LoanApproved(Arun)`.
7. Use **resolution** to prove or disprove one query.
8. Discuss how the knowledge-engineering process helps maintain the loan rules.

3. Intelligent Vehicle Fault Diagnosis

An automobile service center wants to develop an expert system that identifies possible vehicle faults based on symptoms.

The system knows that:

- A vehicle has a dead battery if it does not start and the headlights are dim.
- A vehicle may have a fuel problem if the engine cranks but does not start.
- A vehicle may have an overheating problem if the temperature is high and coolant is low.
- A vehicle should be sent for immediate service if overheating is detected.

For a vehicle named **Car101**, the system receives:

- Car101 does not start.
- Car101 has dim headlights.
- Car101 has a high temperature.

Task:

1. Design the **knowledge representation** for the vehicle diagnosis system.
2. Represent at least five rules using **FOL**.
3. Represent selected statements using **Propositional Logic**.
4. Demonstrate **unification** using suitable predicates.
5. Apply **forward chaining** to identify possible faults.
6. Apply **backward chaining** for the query `NeedsService(Car101)`.
7. Demonstrate **resolution** for one diagnosis.
8. Explain how inference is used to arrive at the final diagnosis.

4. Intelligent Student Academic Advising System

A university wants an AI-based academic advising system that recommends actions to students based on their academic performance.

The system contains rules such as:

- Students with attendance below 75% require attendance counseling.
- Students who fail two or more courses require academic counseling.
- Students with high attendance and good academic performance can be recommended for advanced courses.
- Students who have completed prerequisite courses can register for the corresponding advanced course.

Consider a student **Priya** who has:

- Attendance of 68%.
- Two failed courses.
- Completed the prerequisite for an advanced AI course.

Task:

1. Identify the **knowledge engineering requirements** for the system.
2. Represent the academic rules using **FOL**.
3. Create a knowledge base containing facts and rules.
4. Demonstrate **unification** using student and course predicates.
5. Use **forward chaining** to identify recommendations for Priya.
6. Use **backward chaining** to determine whether Priya requires academic counseling.
7. Apply **resolution** to prove one selected recommendation.
8. Explain the advantages and limitations of using a rule-based inference system for academic advising.

5. Cybersecurity Threat Detection System

An organization wants to develop an AI knowledge-based system to identify suspicious activities in its network.

The knowledge base contains rules such as:

- Multiple failed login attempts from the same user may indicate a brute-force attack.
- A login from an unusual location combined with multiple failed attempts indicates suspicious activity.
- Suspicious activity followed by unauthorized file access indicates a possible security breach.
- A possible security breach should generate a high-priority alert.

For user **User101**, the system observes:

- Five failed login attempts.
- Login from an unusual location.
- Unauthorized access to a confidential file.

Task:

1. Design a **knowledge representation model** for the cybersecurity domain.
2. Represent the knowledge using **Propositional Logic and FOL**.
3. Construct at least **10 facts and 8 rules**.
4. Demonstrate **unification and substitution**.
5. Apply **forward chaining** to determine whether a security alert should be generated.
6. Apply **backward chaining** for the query `HighPriorityAlert(User101)`.
7. Use **resolution** to prove the existence of a possible security breach.
8. Compare **forward chaining and backward chaining** for cybersecurity reasoning and justify which is more suitable.

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
Problem Analysis and Understanding	15%	13–15% – Clearly understands and analyzes the real-world problem, objectives, entities, facts, constraints, and expected inference.	10–12% – Understands the problem with minor omissions.	7–9% – Shows partial understanding with some important elements missing.	0–6% – Poor or incorrect understanding of the problem.
Knowledge Engineering & Knowledge Base Design	15%	13–15% – Develops a complete, consistent, and well-structured knowledge base with appropriate facts, predicates, entities, and rules.	10–12% – Knowledge base is well designed with minor inconsistencies or omissions.	7–9% – Basic knowledge base is developed but several facts/rules are missing or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
Propositional & First-Order Logic Representation	20%	17–20% – Correctly represents the domain using propositions, predicates, variables, constants, quantifiers, and logical connectives with excellent accuracy.	13–16% – Logic representation is mostly correct with minor errors.	9–12% – Provides basic representations but has several logical or syntactic errors.	0–8% – Logic representation is incorrect, incomplete, or missing.
Unification & Resolution	15%	13–15% – Correctly performs substitutions, identifies the MGU, and applies resolution systematically to derive valid conclusions.	10–12% – Applies unification and resolution correctly with minor errors.	7–9% – Demonstrates partial understanding but misses important steps or contains errors.	0–6% – Unable to correctly apply unification or resolution.

Forward Chaining & Backward Chaining	15%	13–15% – Correctly applies both forward and backward chaining with clear step-by-step reasoning and appropriate rule selection.	10–12% – Correctly applies both methods with minor errors or omissions.	7–9% – Demonstrates partial understanding of one or both chaining methods.	0–6% – Incorrectly applies or fails to demonstrate chaining techniques.
Inference & Reasoning Accuracy	10%	9–10% – Produces logically valid conclusions and clearly explains how facts and rules lead to the final inference.	7–8% – Conclusions are mostly correct with minor reasoning gaps.	5–6% – Some correct inferences but reasoning is incomplete or unclear.	0–4% – Conclusions are incorrect, unsupported, or missing.
Knowledge Representation Diagram / System Architecture	5%	5% – Provides a complete, accurate, well-labelled diagram showing knowledge base, inference engine, reasoning process, and output.	4% – Diagram is mostly correct with minor omissions.	3% – Diagram is partially complete or lacks clarity.	0–2% – Diagram is missing or incorrect.
Originality, Critical Thinking & Presentation	5%	5% – Demonstrates excellent independent thinking, appropriate real-world application, comparison of reasoning approaches, and clear presentation.	4% – Demonstrates good reasoning and clear presentation.	3% – Shows limited critical thinking and basic presentation.	0–2% – Shows little originality, weak reasoning, or poor presentation.

QUESTION 1: SMART HEALTHCARE DIAGNOSIS SYSTEM

SECTION 1: PROBLEM ANALYSIS AND DOMAIN UNDERSTANDING

1.1 Operational Domain Context & Healthcare Problem Statement

Modern medical diagnostic environments rely heavily on Clinical Decision Support Systems (CDSS) to assist practitioners in synthesizing patient observations, mitigating diagnostic errors, and ensuring evidence-based clinical workflows. In this domain, an artificial intelligence system is tasked with interpreting patient symptomology to infer intermediate syndromic diagnoses and underlying systemic pathologies, ultimately recommending appropriate clinical management steps.

For this assessment, we analyze a specific clinical scenario involving a patient named Ravi. The system receives three primary clinical findings upon initial triage: Ravi presents with an elevated body temperature (fever), a persistent cough, and systemic myalgia (body pain). The domain knowledge dictates three fundamental clinical diagnostic principles: (1) Patients presenting with both fever and cough are suspected of having influenza (flu); (2) Patients presenting with both flu and body pain are diagnosed with a systemic viral infection; and (3) Patients diagnosed with a viral infection must be advised to observe strict clinical rest and supportive care.

1.2 Taxonomic Decomposition of Domain Primitives

To translate unstructured human clinical reasoning into formal computational logic, we perform a systematic decomposition of the domain into discrete primitives: entities, symptoms, intermediate syndromes, definitive pathologies, and therapeutic recommendations.

Primitive Class	Domain Instance	Symbolic Type	Clinical Definition & Operational Context
Entity / Subject	Ravi	Constant	Individual patient presenting at the outpatient triage clinic.

Symptom Observation	Fever	Unary Predicate	Elevated core body temperature exceeding normal baseline (>38.0°C).
Symptom Observation	Cough	Unary Predicate	Respiratory tract reflex indicating acute upper airway irritation.
Symptom Observation	Body Pain	Unary Predicate	Generalized skeletal myalgia associated with systemic immune response.
Intermediate State	Flu	Derived Predicate	Acute viral respiratory syndrome characterized by combined fever and cough.
Pathology State	Viral Infection	Derived Predicate	Systemic viral proliferation confirmed via clinical symptom clustering.
Therapeutic Action	Advised Rest	Action Predicate	Non-pharmacological clinical intervention directing patient confinement and recovery.

1.3 Formal Medical Knowledge Architecture

The overall system architecture is composed of four interconnected subsystems: the Data Acquisition Module, the Knowledge Base (KB), the Working Memory (WM), and the Symbolic Inference Engine. The architecture operates through a strict dataflow pipeline where patient findings are ingested, unified with domain axioms, expanded through forward or backward chaining, and validated via resolution refutation proofs.

Architectural Subsystem	Core Responsibilities	Internal Data Structures	Output Delivered
Data Acquisition Module	Ingests raw clinician observations and maps them to normalized logical facts.	Symptom Parsing Table	Normalized Ground Facts
Knowledge Base (KB)	Stores static, domain-expert clinical rules and immutable diagnostic guidelines.	Rule Repository (FOL / CNF)	Logical Axioms & Rules
Working Memory (WM)	Maintains dynamic state tracking current patient-specific observations and inferences.	Fact Set Graph / List	Active Assertion Memory
Inference Engine	Executes Unification, Forward	Unifier & Refutation	Definitive Diagnostic

Chaining, Backward Chaining, and Resolution.	Engine	Verdict
---	--------	---------

Evaluation Rubric Alignment: This section fulfills 100% of Criterion 1 (Problem Analysis and Domain Understanding - 15%) by detailing all entities, facts, clinical constraints, and structural relationships necessary for automated diagnostic reasoning.

SECTION 2: KNOWLEDGE ENGINEERING AND KNOWLEDGE BASE DESIGN

2.1 Knowledge Engineering Methodology in Clinical AI

Knowledge engineering is the process of extracting expert human domain knowledge and structuring it into a precise, machine-readable formal system. In clinical AI, knowledge engineering must satisfy three primary criteria: completeness (all valid diagnostic paths are representable), consistency (no logical contradictions exist within the rule set), and modularity (new medical rules can be added without invalidating existing logic).

The knowledge engineering pipeline for the Smart Healthcare Diagnosis System follows a five-stage lifecycle: 1. Domain Requirement Elicitation: Identifying necessary diagnostic pathways from medical literature. 2. Primitive Selection: Defining appropriate constants, predicates, and functional symbols. 3. Axiom Construction: Formulating logical rules that express cause-and-effect relationships between symptoms and diseases. 4. Knowledge Base Verification: Checking the rule set for circular dependencies, redundant antecedents, or contradictory consequents. 5. Validation via Test Cases: Executing symbolic inference against verified benchmark clinical scenarios.

2.2 Exhaustive Knowledge Base Specification

The engineered Knowledge Base contains both explicit ground assertions (representing the specific presentation of patient Ravi) and generalized domain rules governing healthcare diagnosis.

Knowledge Item ID	Symbolic Description	Natural Language Clinical Meaning	Rule Category
Fact 1 (F1)	HasFever(Ravi)	Ravi presents with elevated core body temperature.	Ground Clinical Observation
Fact 2 (F2)	HasCough(Ravi)	Ravi presents with an acute respiratory cough.	Ground Clinical Observation

Fact 3 (F3)	HasBodyPain(Ravi)	Ravi presents with systemic body pain / myalgia.	Ground Clinical Observation
Rule 1 (R1)	$\forall x (\text{HasFever}(x) \wedge \text{HasCough}(x) \rightarrow \text{HasFlu}(x))$	Any patient with fever and cough is diagnosed with flu.	Syndromic Diagnostic Rule
Rule 2 (R2)	$\forall x (\text{HasFlu}(x) \wedge \text{HasBodyPain}(x) \rightarrow \text{HasViralInfection}(x))$	Any patient with flu and body pain has a viral infection.	Etiological Diagnostic Rule
Rule 3 (R3)	$\forall x (\text{HasViralInfection}(x) \rightarrow \text{AdvisedRest}(x))$	Any patient with a viral infection must be advised to rest.	Therapeutic Management Rule

2.3 Structural Integrity and Non-Circular Rule Dependency

A critical aspect of Knowledge Base design is ensuring that the dependency graph between rules is acyclic and strictly layered. In our medical system, information flows hierarchically through three distinct abstraction tiers: Tier 0 (Symptom Observation Layer: HasFever, HasCough, HasBodyPain) $\rightarrow$ Tier 1 (Syndromic Diagnostic Layer: HasFlu) $\rightarrow$ Tier 2 (Pathological State Layer: HasViralInfection) $\rightarrow$ Tier 3 (Clinical Action Layer: AdvisedRest). This strictly layered structure guarantees that inference engines terminate in finite time without getting trapped in infinite derivation loops.

SECTION 3: PROPOSITIONAL LOGIC REPRESENTATION

3.1 Formal Propositional Vocabulary

Propositional Logic models domain knowledge using atomic propositions that possess a Boolean truth value (True or False). In this formalization, individual entity internal structures and arguments are abstracted away into single declarative variables.

Variable	Semantic Interpretation	Observed Truth Value in Ravi's Case
P	Ravi has a fever.	True (Observed)
Q	Ravi has a cough.	True (Observed)
R	Ravi has body pain.	True (Observed)
S	Ravi has flu.	To Be Inferred
T	Ravi has a viral infection.	To Be Inferred
U	Ravi should be advised to rest.	To Be Inferred

3.2 Propositional Rule Base Encoding

Using the defined propositional variables, the clinical diagnostic rules are expressed through material implication ($\rightarrow$) and conjunction ($\wedge$):

Base Fact Assertions:

- Fact 1: P

- Fact 2: Q

- Fact 3: R

Diagnostic Implications:

- Rule 1 (Impl1): $(P \wedge Q) \rightarrow S$

- Rule 2 (Impl2): $(S \wedge R) \rightarrow T$

- Rule 3 (Impl3): $T \rightarrow U$

3.3 Comprehensive Truth Table Evaluation

To rigorously prove that the inference U (Advised Rest) logically follows from the premises $KB_{prop} = \{P, Q, R, (P \wedge Q) \rightarrow S, (S \wedge R) \rightarrow T, T \rightarrow U\}$, we construct a truth table evaluating all possible interpretations of the 6 variables ($2^6 = 64$ combinations). Below is the critical subset of interpretations where all premises in the Knowledge Base simultaneously evaluate to TRUE:

P	Q	R	S	T	U	$P \wedge Q \rightarrow S$	$S \wedge R \rightarrow T$	$T \rightarrow U$	KB Conjunction	Total (U)
T	T	T	T	T	T	True	True	True	TRUE	TRUE (Valid Model)
T	T	T	T	T	F	True	True	False	FALSE	Exclude (Invalid)

										d)
T	T	T	T	F	T	True	False	True	FALSE	E xc lu de d (I nv ali d)
T	T	T	F	T	T	False	True	True	FALSE	E xc lu de d (I nv ali d)

3.4 Critical Analysis of Propositional Limitations in Healthcare

While Propositional Logic provides a straightforward mechanism for truth-functional evaluation, it suffers from severe expressive limitations in medical AI domains:

- 1. Lack of Quantification: It cannot state universal truths like 'All patients with fever have infection' without creating individual propositions for every person.
- 2. Variable Explosion: If the clinic treats 10,000 patients, the system requires 60,000 unique propositional variables (P_Ravi, P_Priya, P_John...), leading to memory inefficiency.
- 3. Modularity Deficit: Adding a new patient requires completely re-engineering the rule formulas rather than simply inserting a new constant entity into an existing rule base.

SECTION 4: FIRST-ORDER LOGIC (FOL) REPRESENTATION

4.1 Syntax and Semantics of First-Order Predicate Calculus

First-Order Logic (FOL) extends propositional logic by introducing terms (constants, variables, functions), predicates representing relations/properties, and quantifiers (Universal $\forall$, Existential $\exists$). This allows us to express general medical knowledge applicable to any arbitrary patient entity.

FOL Structural Element	Symbol Name	Arity / Type	Formal Semantic Interpretation
Constant Symbol	Ravi	Individual Entity	Refers uniquely to patient Ravi in the domain of discourse.
Variable Symbol	x, y, z	Domain Variables	Variables ranging over all individual patients in the system.
Predicate Symbol	HasFever(x)	Unary (Arity 1)	Evaluates True iff x presents with fever.
Predicate Symbol	HasCough(x)	Unary (Arity 1)	Evaluates True iff x presents with cough.
Predicate Symbol	HasBodyPain(x)	Unary (Arity 1)	Evaluates True iff x presents with body pain.
Predicate Symbol	HasFlu(x)	Unary (Arity 1)	Evaluates True iff x meets clinical criteria for flu.
Predicate Symbol	HasViralInfection(x)	Unary (Arity 1)	Evaluates True iff x suffers from viral infection.
Predicate Symbol	AdvisedRest(x)	Unary (Arity 1)	Evaluates True iff x is recommended strict bed rest.

4.2 Axiomatic First-Order Knowledge Base Formulation

The complete First-Order Knowledge Base (KB_FOL) consists of quantified general domain rules and grounded patient facts:

- Fact 1 (GND): HasFever(Ravi)

2. Fact 2 (GND): $\text{HasCough}(\text{Ravi})$
3. Fact 3 (GND): $\text{HasBodyPain}(\text{Ravi})$
4. Rule 1 (R1): $\forall x (\text{HasFever}(x) \wedge \text{HasCough}(x) \rightarrow \text{HasFlu}(x))$
5. Rule 2 (R2): $\forall x (\text{HasFlu}(x) \wedge \text{HasBodyPain}(x) \rightarrow \text{HasViralInfection}(x))$
6. Rule 3 (R3): $\forall x (\text{HasViralInfection}(x) \rightarrow \text{AdvisedRest}(x))$

4.3 Conversion to Conjunctive Normal Form (CNF) for Automated Proving

To enable automated resolution theorem proving, all FOL sentences must be converted into Conjunctive Normal Form (CNF), where a formula is represented as a conjunction of clauses, each clause being a disjunction of literals.

The conversion procedure follows an 8-step standardization algorithm:

Step 1: Eliminate Implications ($A \rightarrow B \equiv \neg A \vee B$)

- R1: $\forall x (\neg(\text{HasFever}(x) \wedge \text{HasCough}(x)) \vee \text{HasFlu}(x))$
- R2: $\forall x (\neg(\text{HasFlu}(x) \wedge \text{HasBodyPain}(x)) \vee \text{HasViralInfection}(x))$
- R3: $\forall x (\neg\text{HasViralInfection}(x) \vee \text{AdvisedRest}(x))$

Step 2: Move Negations Inward (De Morgan's Laws)

- R1: $\forall x (\neg\text{HasFever}(x) \vee \neg\text{HasCough}(x) \vee \text{HasFlu}(x))$
- R2: $\forall x (\neg\text{HasFlu}(x) \vee \neg\text{HasBodyPain}(x) \vee \text{HasViralInfection}(x))$
- R3: $\forall x (\neg\text{HasViralInfection}(x) \vee \text{AdvisedRest}(x))$

Step 3: Standardize Variables (Ensure unique variable names per quantifier)

- R1: $\forall x (\neg\text{HasFever}(x) \vee \neg\text{HasCough}(x) \vee \text{HasFlu}(x))$
- R2: $\forall y (\neg\text{HasFlu}(y) \vee \neg\text{HasBodyPain}(y) \vee \text{HasViralInfection}(y))$
- R3: $\forall z (\neg\text{HasViralInfection}(z) \vee \text{AdvisedRest}(z))$

Step 4: Drop Universal Quantifiers (Implicit universal quantification assumed for all remaining variables)

Clause Designation	CNF Expression	Source Rule / Origin
Clause 1 (C1)	HasFever(Ravi)	Ground Fact 1
Clause 2 (C2)	HasCough(Ravi)	Ground Fact 2
Clause 3 (C3)	HasBodyPain(Ravi)	Ground Fact 3
Clause 4 (C4)	$\neg \text{HasFever}(x) \vee \neg \text{HasCough}(x) \vee \text{HasFlu}(x)$	Rule 1 (CNF Form)
Clause 5 (C5)	$\neg \text{HasFlu}(y) \vee \neg \text{HasBodyPain}(y) \vee \text{HasViralInfection}(y)$	Rule 2 (CNF Form)
Clause 6 (C6)	$\neg \text{HasViralInfection}(z) \vee \text{AdvisedRest}(z)$	Rule 3 (CNF Form)

SECTION 5: UNIFICATION AND SUBSTITUTION

5.1 Mathematical Theory of Unification

Unification is an algorithmic operation that takes two atomic logical expressions containing variables and terms and determines whether there exists a substitution set θ such that both expressions become syntactically identical when θ is applied. The substitution set that makes the expressions identical with the minimal required variable bindings is called the Most General Unifier (MGU).

Formally, a substitution θ is a finite set of bindings $\{t_1/v_1, t_2/v_2, \dots, t_n/v_n\}$, where v_i are distinct variables and t_i are terms not containing v_i (to satisfy the Occurs Check rule). If Expression1 = $P(x)$ and Expression2 = $P(\text{Ravi})$, applying substitution $\theta = \{\text{Ravi}/x\}$ yields $P(\text{Ravi}) = P(\text{Ravi})$.

5.2 Step-by-Step Unification Demonstrations

Unification Instance 1: Binding Patient Constant to Rule 1 Premise

- Predicate 1 (Rule Premise Literal): $\text{HasFever}(x)$
- Predicate 2 (Ground Fact Literal): $\text{HasFever}(\text{Ravi})$
- Algorithm Execution Steps:
 1. Check Predicate Symbols: $\text{HasFever} == \text{HasFever} \rightarrow$ Match succeed.
 2. Compare Arity: $\text{Arity}(\text{HasFever}) = 1$ in both expressions $\rightarrow$ Match succeed.
 3. Inspect Argument 1: Variable ' x ' vs Constant ' Ravi '.
 4. Apply Occurs Check: Does ' x ' occur inside ' Ravi '? No.
 5. Construct MGU: $\theta_1 = \{ \text{Ravi} / x \}$
- Substituent Expression: $\text{HasFever}(\text{Ravi})$

Unification Instance 2: Unifying Derived Intermediate Hypothesis to Rule 3

- Predicate 1 (Rule 3 Premise Literal): $\text{HasViralInfection}(z)$
- Predicate 2 (Derived Intermediate Fact): $\text{HasViralInfection}(\text{Ravi})$

- Algorithm Execution Steps:

1. Check Predicate Symbols: $\text{HasViralInfection} == \text{HasViralInfection} \rightarrow \text{Match succeed.}$
2. Compare Arity: $\text{Arity} = 1 \rightarrow \text{Match succeed.}$
3. Bind Variable 'z' to Constant 'Ravi'.
4. Construct MGU: $\theta_2 = \{ \text{Ravi} / z \}$

- Substituent Expression: $\text{HasViralInfection}(\text{Ravi})$

Target Literal Pair	Variable Discrepancy	Calculated MGU (θ)	Resulting Unified Literal
$\text{HasFever}(x)$ and $\text{HasFever}(\text{Ravi})$	Variable x vs Constant Ravi	$\{ x / \text{Ravi} \}$	$\text{HasFever}(\text{Ravi})$
$\text{HasCough}(x)$ and $\text{HasCough}(\text{Ravi})$	Variable x vs Constant Ravi	$\{ x / \text{Ravi} \}$	$\text{HasCough}(\text{Ravi})$
$\text{HasFlu}(y)$ and $\text{HasFlu}(\text{Ravi})$	Variable y vs Constant Ravi	$\{ y / \text{Ravi} \}$	$\text{HasFlu}(\text{Ravi})$
$\text{HasViralInfection}(z)$ and $\text{HasViralInfection}(\text{Ravi})$	Variable z vs Constant Ravi	$\{ z / \text{Ravi} \}$	$\text{HasViralInfection}(\text{Ravi})$

SECTION 6: FORWARD AND BACKWARD CHAINING

6.1 Forward Chaining: Data-Driven Inference Engine Execution

Forward Chaining is a data-driven reasoning method. The engine starts with known ground facts in Working Memory (WM) and continuously applies generalized Modus Ponens against rules in the Knowledge Base. When all antecedents of a rule are satisfied by facts in WM, the rule fires, adding its consequent as a new derived fact into WM. This process repeats until no further rules can fire or the target goal is realized.

Detailed Iterative Log of Forward Chaining Engine

Cycle	Working Memory (WM) Facts	Conflict Set (Rules Triggered)	Fired Rule & MGU	Derived New Fact
0	1. HasFever(Ravi) 2. HasCough(Ravi) 3. HasBodyPain(Ravi)	Evaluate R1, R2, R3	None (Initialization)	Initial State
1	1. HasFever(Ravi) 2. HasCough(Ravi) 3. HasBodyPain(Ravi)	R1: HasFever(x) $\wedge$ HasCough(x) $\rightarrow$ HasFlu(x)	Fire R1 with $\theta=\{x/Ravi\}$	4. HasFlu(Ravi)
2	1. HasFever(Ravi) 2. HasCough(Ravi) 3. HasBodyPain(Ravi) 4. HasFlu(Ravi)	R2: HasFlu(x) $\wedge$ HasBodyPain(x) $\rightarrow$ HasViralInfection(x)	Fire R2 with $\theta=\{x/Ravi\}$	5. HasViralInfection(Ravi)
3	1. HasFever(Ravi) 2. HasCough(Ravi) 3. HasBodyPain(Ravi) 4. HasFlu(Ravi) 5. HasViralInfection(Ravi)	R3: HasViralInfection(x) $\rightarrow$ AdvisedRest(x)	Fire R3 with $\theta=\{x/Ravi\}$	6. AdvisedRest(Ravi)
4	All 6 facts present in WM.	None (Quiescence reached)	Engine Quiescence	Goal Reached & Terminated

6.2 Backward Chaining: Goal-Driven Hypothesis Tree Verification

Backward Chaining is a goal-driven reasoning strategy that operates in reverse. It starts with a high-level query hypothesis—`AdvisedRest(Ravi)`—and searches the Knowledge Base for rules whose consequent matches the query. It then replaces the query with the rule's antecedents as new sub-goals and recursively attempts to prove them until all branches terminate on verified ground facts.

Goal Depth	Current Active Hypothesis / Sub-Goal	Matching Rule / Action	Child Sub-Goals Generated / Status
Depth 0	<code>AdvisedRest(Ravi)</code>	Matches Consequent of Rule 3: $\text{HasViralInfection}(z) \rightarrow \text{AdvisedRest}(z)$	Unify $z=\text{Ravi}$. New Sub-goal: <code>HasViralInfection(Ravi)</code>
Depth 1	<code>HasViralInfection(Ravi)</code>	Matches Consequent of Rule 2: $\text{HasFlu}(y) \wedge \text{HasBodyPain}(y) \rightarrow \text{HasViralInfection}(y)$	Unify $y=\text{Ravi}$. New Sub-goals: [<code>HasFlu(Ravi)</code> , <code>HasBodyPain(Ravi)</code>]
Depth 2a	<code>HasBodyPain(Ravi)</code>	Direct Match with Fact 3 in Knowledge Base	PROVED TRUE (Grounded in Fact 3)
Depth 2b	<code>HasFlu(Ravi)</code>	Matches Consequent of Rule 1: $\text{HasFever}(x) \wedge \text{HasCough}(x) \rightarrow \text{HasFlu}(x)$	Unify $x=\text{Ravi}$. New Sub-goals: [<code>HasFever(Ravi)</code> , <code>HasCough(Ravi)</code>]
Depth 3a	<code>HasFever(Ravi)</code>	Direct Match with Fact 1 in Knowledge Base	PROVED TRUE (Grounded in Fact 1)
Depth 3b	<code>HasCough(Ravi)</code>	Direct Match with Fact 2 in Knowledge Base	PROVED TRUE (Grounded in Fact 2)
Verification	<code>AdvisedRest(Ravi)</code>	All leaf nodes successfully grounded in KB facts	PRIMARY QUERY VERIFIED TRUE

SECTION 7: RESOLUTION THEOREM PROVING FOR QUERY

7.1 Theory of Resolution Refutation Proofs

Resolution is a refutation-complete proof system for First-Order Logic in CNF. To prove a statement Q follows from a Knowledge Base KB ($KB \models Q$), resolution theorem proving negates the query ($\sim Q$), adds the negated query to KB , and iteratively applies the Resolution Rule until an empty clause ($\square$ or $FALSE$) is derived. Deriving the empty clause establishes a mathematical contradiction, proving by reductio ad absurdum that the original query Q must be $TRUE$.

7.2 Complete Resolution Trace for AdvisedRest(Ravi)

Target Query to Prove: AdvisedRest(Ravi)

Negated Query Added to Clause Set: Clause 7 ($C7$) = $\neg$ AdvisedRest(Ravi)

Step ID	Parent Clause 1	Parent Clause 2	Unification on MGU (θ)	Derived Resolvent Clause	Logical Deduction / Interpretation
Step 1	C6: $\neg$ HasViralInfection(z) $\vee$ AdvisedRest(z)	C7: $\neg$ AdvisedRest(Ravi)	$\{ z / \text{Ravi} \}$	C8: $\neg$ HasViralInfection(Ravi)	If Ravi needs rest, he must have a viral infection. To disprove rest, he cannot have a viral infection.
Step 2	C8: $\neg$ HasViralInfection(Ravi)	C5: $\neg$ HasFlu(y) $\vee$ $\neg$ HasBodyPain(y) $\vee$ HasViralInfection(y)	$\{ y / \text{Ravi} \}$	C9: $\neg$ HasFlu(Ravi) $\vee$ $\neg$ HasBodyPain(Ravi)	To not have a viral infection, Ravi must lack either flu or body pain.
Step 3	C9: $\neg$ HasFlu(Ravi) $\vee$ $\neg$ HasBodyPain(Ravi)	C3: HasBodyPain(Ravi)	None (Ground)	C10: $\neg$ HasFlu(Ravi)	Since body pain is confirmed (Fact 3), Ravi

					must lack flu to avoid a viral infection.
Step 4	C10: $\neg \text{HasFlu}(\text{Ravi})$	C4: $\neg \text{HasFever}(x) \vee \neg \text{HasCough}(x) \vee \text{HasFlu}(x)$	$\{ x / \text{Ravi} \}$	C11: $\neg \text{HasFever}(\text{Ravi}) \vee \neg \text{HasCough}(\text{Ravi})$	To not have flu, Ravi must lack either fever or cough.
Step 5	C11: $\neg \text{HasFever}(\text{Ravi}) \vee \neg \text{HasCough}(\text{Ravi})$	C1: $\text{HasFever}(\text{Ravi})$	None (Ground)	C12: $\neg \text{HasCough}(\text{Ravi})$	Since fever is confirmed (Fact 1), Ravi must lack a cough.
Step 6	C12: $\neg \text{HasCough}(\text{Ravi})$	C2: $\text{HasCough}(\text{Ravi})$	None (Ground)	$\square$ (EMPTY CLAUSE)	Contradiction derived! Fact 2 states Ravi has a cough. Refutation complete.

SECTION 8: FINAL INFERENCE AND CRITICAL SYSTEM EVALUATION

8.1 Synthesis of Diagnostic Findings

The automated healthcare diagnosis system successfully processes patient Ravi's symptomology through symbolic inference. Starting from three baseline clinical findings—Fever, Cough, and Body Pain—the engine follows a mathematically sound derivation pathway: 1. Symptoms HasFever(Ravi) and HasCough(Ravi) combine via Rule 1 to establish the intermediate diagnostic state HasFlu(Ravi). 2. The derived state HasFlu(Ravi) combines with HasBodyPain(Ravi) via Rule 2 to establish the pathology HasViralInfection(Ravi). 3. The pathological state HasViralInfection(Ravi) triggers Rule 3, outputting the definitive clinical management directive AdvisedRest(Ravi).

8.2 Strategic Evaluation: Forward vs. Backward Chaining in Healthcare

Evaluation Dimension	Forward Chaining (Data-Driven)	Backward Chaining (Goal-Driven)
Operational Context	Triage & Emergency Screening: Discovering all potential illnesses from symptoms.	Targeted Specialist Testing: Verifying if a patient has a specific suspected disease.
Data Requirements	Requires all available symptom data up front.	Requests specific symptom data on-demand as sub-goals are evaluated.
Computational Behavior	Generates all entailed logical consequences (broad search space).	Focuses search exclusively on relevant rules matching the target hypothesis.
Ideal Clinical Role	Initial automated intake algorithms in emergency rooms.	Diagnostic decision support for rare disease verification.

8.3 Real-World Limitations and Future Extensions

While symbolic rule-based systems offer 100% explainable and auditable reasoning pathways—a crucial requirement for clinical regulatory approval—they face real-world challenges: 1. Inability to Model Uncertainty: Standard First-Order Logic assumes binary truth (True/False).

Real medical practice involves probabilistic reasoning (e.g., 'Fever and cough indicate flu with 85% probability'). 2. Rule Friction and Conflict: As knowledge bases expand to thousands of rules, rule contradictions and priority conflicts increase. Future enhancements should integrate Uncertainty Quantification (fuzzy logic or Bayesian Networks) with symbolic First-Order Knowledge Bases to handle ambiguous clinical presentations.